

Manual de operación

Simulación conjunta VaN3Twin + SUMO-GEO

Ejecutar una simulación V2X de ns-3 visualizada en 3D y en vivo en el navegador

Universidad de Cuenca — Redes Vehiculares y Heterogéneas (INGE-00104)

Prof. Pablo Barbecho — 25 de agosto de 2026

Contenido

1	Qué hace esta integración	1
2	Componentes, ubicaciones y puertos	2
3	Requisitos previos (ya aplicados)	2
4	Modos de ejecución y arranque	3
5	Ejecutar una simulación con el visor (el orden importa)	4
6	Flags útiles del ejemplo EVA	6
7	Velocidad de la simulación en el visor	6
8	Qué ofrece el visor durante la corrida	7
9	Editar código y recompilar	7
10	Replay offline: mensajes V2X desde los .pcap	8
11	Solución de problemas	9
11.1	Comandos de diagnóstico, en detalle	9
11.2	Cuándo usar <code>--build</code> (y cuándo no)	10
12	Cierre de sesión	11
13	Resumen de comandos	11

1 Qué hace esta integración

VaN3Twin (framework V2X sobre ns-3) y SUMO-GEO (visor web 3D MapLibre/deck.gl) se acoplan a la **misma** instancia de SUMO mediante TraCI *multi-cliente*: ns-3 es el cliente 1 (controla y marca el ritmo) y el backend de SUMO-GEO es el cliente 2 (solo lectura). SUMO avanza en *lockstep* — exige que ambos clientes pidan paso — pero el visor pide objetivos gruesos ($t + 1$ s), de modo que **no frena** a ns-3.

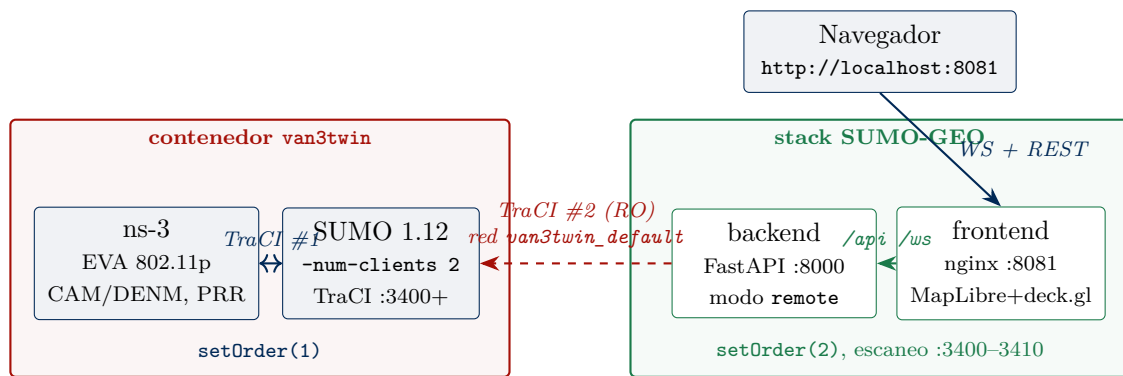


Figure 1: Arquitectura validada: dos proyectos Docker Compose unidos por la red `van3twin_default`; la geometría del mapa se lee del volumen `van3twin_ns3-workspace`.

2 Componentes, ubicaciones y puertos

Pieza	Dónde	Rol
Stack van3twin	<code>~/van3twin-docker</code>	Contenedor <code>van3twin: ns-3/VaN3Twin + SUMO 1.12.0</code>
Árbol ns-3 compilable	volumen <code>van3twin_ns3-workspace</code>	Único lugar donde se compila (case-sensitive)
Edición de código ns-3	dentro del contenedor (<code>docker compose exec</code> o VS Code Dev Containers)	Se edita directamente en el volumen
Repo SUMO-GEO	<code>~/Dropbox/Mac/Documents/SUMO_GEO</code>	Backend FastAPI + frontend MapLibre/deck.gl
Servicios del visor	profile visor del mismo <code>docker-compose.yml</code> de van3twin	backend (remote) + frontend nginx; solo arrancan con <code>--profile visor</code>

Puerto (Mac)	Servicio
8081	Visor SUMO-GEO (frontend nginx)
8000	API del backend SUMO-GEO (<code>/api/health</code> , <code>/api/meta...</code>)
8080	vehicle-visualizer nativo de VaN3Twin (Leaflet 2D)
5901	VNC del contenedor (GUI de SUMO)
8090	GEO SUMO WEB de la práctica (percepción por radio)
3400+	TraCI — no publicado : solo red Docker interna

3 Requisitos previos (ya aplicados)

El acoplamiento depende de estos parches, ya presentes y compilados. Verificar solo si algo falla:

1. **traci-client.cc** (ns-3): `NS_OBJECT_ENSURE_REGISTERED(TraciClient)` y llamada a `setOrder(1)` tras el `connect` cuando las opciones de SUMO contienen `--num-clients`.
2. **Ejemplo EVA** (`v2v-emergencyVehicleAlert-80211p.cc`): flag `--num-traci-clients=N` que añada `--num-clients N` al comando de SUMO.
3. **Backend SUMO-GEO**: `sumo_order/sumo_port_scan` en la configuración; `setOrder(2)` y escaneo de puertos 3400–3410 al conectar; paso con objetivo absoluto; detección de proyección compatible con `sumolib 1.12 (geo.py)`.
4. **Servicios backend/frontend** (profile visor del compose de van3twin): `APP_SUMO_HOST=van3twin`,

APP_SUMO_PORT=3400, APP_SUMO_PORT_SCAN=10, APP_SUMO_ORDER=2, y el net del EVA leído del volumen en solo lectura.

■ EN EL MAC (PC local) — comprobación rápida

```
cd ~/van3twin-docker
docker compose exec van3twin grep -c "num-clients" src/traci/model/traci-client.cc # >= 1
```

4 Modos de ejecución y arranque

Todo vive en un solo compose (`~/van3twin-docker/docker-compose.yml`). Dos cosas independientes definen el modo de operación: el **profile** decide qué contenedores existen, y el flag `--num-traci-clients` decide, por corrida, si SUMO espera al visor. Van siempre en pareja (figura 2).

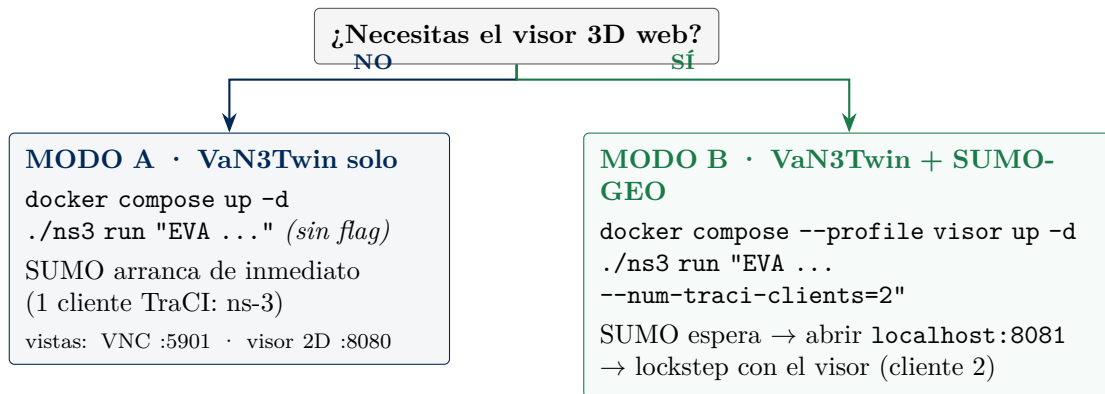
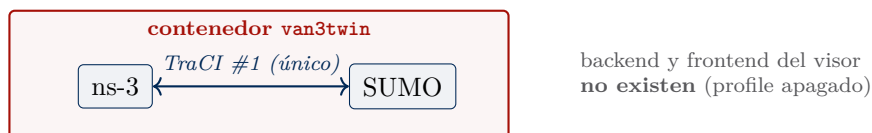


Figure 2: Elección del modo de operación: el profile crea (o no) los contenedores del visor; el flag hace que SUMO espere (o no) al segundo cliente TraCI.

Advertencia

No levantar a la vez el `docker-compose.yml` normal del repo SUMO_GEO (modo *managed*, para uso autónomo del visor): comparte los puertos 8000/8081 con el Modo B y provocaría conflictos.

Modo A — VaN3Twin solo (uso clásico)



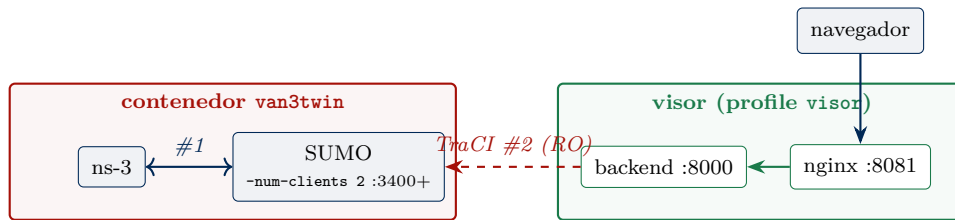
■ EN EL MAC (PC local)

```
cd ~/van3twin-docker && docker compose up -d # sin --profile: el visor no arranca
docker compose exec van3twin bash
```

⚡ DENTRO DEL CONTENEDOR van3twin

```
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --met-sup=true"
```

Sin `--num-traci-clients`: SUMO no espera a nadie y la simulación arranca de inmediato. Opcionales: GUI de SUMO por VNC (open `vnc://127.0.0.1:5901` en el Mac y quitar `--sumo-gui=false`) o el visor 2D nativo (`--vehicle-visualizer=true` → `:8080`).

Modo B — VaN3Twin + visor 3D SUMO-GEO (profile visor)**■ EN EL MAC (PC local)**

```
cd ~/van3twin-docker && docker compose --profile visor up -d
```

y seguir la §5 (Pasos A–B–C) lanzando el EVA con `--num-traci-clients=2`. Flag y visor van siempre juntos: con el flag SUMO espera al visor; sin el flag, el visor no puede conectar (pantalla en negro).

Nota

Se puede alternar entre modos sin tocar contenedores: si el visor ya está arriba pero se lanza sin el flag, la simulación corre sola (Modo A) y el visor simplemente no conecta en esa corrida.

Mantenimiento del stack**Nota: cuándo usar --build**

`docker compose --profile visor build backend + up -d` solo si cambió algo de backend/ del repo SUMO_GEO (código Python, requirements o Dockerfile). Uso diario ⇒ nada: el stack queda corriendo en segundo plano.

5 Ejecutar una simulación con el visor (el orden importa)**Paso A — limpiar restos de la corrida anterior****⚡ DENTRO DEL CONTENEDOR van3twin**

```
cd ~/van3twin-docker && docker compose exec van3twin bash
# dentro del contenedor:
pkill -f ns3-dev-v2v; pkill sumo; sleep 1      # inofensivo si no hay nada
```

Paso B — lanzar el EVA pidiendo 2 clientes TraCI

❏ DENTRO DEL CONTENEDOR `van3twin`

```
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --met-sup=true --num-traci-clients=2"
```

Salida esperada:

```
***Starting server on port 34XX ***
waiting for 2 clients...
client connected
```

...y ahí **se queda esperando**. Es el comportamiento correcto: SUMO no arranca hasta que conecte el visor (cliente 2).

Nota: el puerto puede variar

El puerto puede ser 3400, 3401, 3402...: el cliente TraCI de ns-3 (`GetFreePort`) salta al siguiente puerto libre si el anterior quedó en estado `TCP TIME_WAIT` (≈ 60 s tras cada corrida). El backend escanea 3400–3410 y lo encuentra solo; no hay que configurar nada.

Paso C — abrir (o recargar) el visor

✂ EN EL NAVEGADOR

Abrir <http://localhost:8081>. Al conectar el WebSocket, en el terminal aparece el segundo `client connected` seguido de `Simulation version 1.12.0 started`: todo corre en lockstep. El visor abre sobre **Turín** con los 7–8 vehículos del EVA.

Validado empíricamente (25-ago-2026)

Secuencia completa observada en producción: `waiting for 2 clients` → conectan ns-3 (orden 1) y el backend (orden 2) → lockstep sin frenar a ns-3 → visor 3D en vivo; `/api/health` responde `{"status":"ok","mode":"remote"}`.

Reglas de la sesión

- **Un visor por corrida.** No recargar la página a mitad de simulación ni abrir una segunda pestaña: SUMO fijó 2 clientes al arrancar. Tras una recarga, volver al Paso A.
- No usar el selector Bajo/Medio/Alto del visor (pertenece al modo *managed*).
- Al terminar (`--sim-time` o fin de rutas), VaN3Twin cierra SUMO (`--quit-on-end`) y el visor pierde la conexión: es lo esperado. ns-3 imprime su resumen (CAMs, PRR, etc.).
- Nueva corrida = Paso A → B → C. El stack SUMO-GEO sigue arriba; no se toca.

6 Flags útiles del ejemplo EVA

Flag	Efecto
--num-traci-clients=2	Obligatorio para el visor: SUMO espera 2 clientes TraCI
--sim-time=250	Duración de la simulación (s)
--csv-log=nombre	Log CSV por vehículo de la aplicación EVA
--sumo-updates=0.1	Sincronización TraCI cada 0.1 s sim (default 0.01) → ~10× menos overhead, acelera el lockstep
--met-sup=true	Metric supervisor (PRR, latencia)
--vehicle-visualizer=true	Visor 2D nativo además del 3D (:8080)
--send-cam=false	Desactiva la transmisión de CAMs (escenario de referencia)
--penetrationRate=0.5	Fracción de vehículos con equipo V2X

Advertencia

Sin espacios dentro de cada flag: el `./ns3 run` de este fork parte la cadena por espacios *sin respetar comillas internas*.

7 Velocidad de la simulación en el visor

El slider **Velocidad (fps)** de la web sí actúa: cada movimiento envía `{cmd:"speed"}` al backend, que pide a SUMO un avance de `APP_STEP_LENGTH` (1 s simulado) por frame. El techo del visor es por tanto `fps × 1 s`: a 10 fps hasta 10× tiempo real, a 30 fps hasta 30×. Pero en *lockstep* manda el cliente más lento, y suele ser **ns-3**: el EVA sincroniza TraCI cada `sumo_updates = 0,01 s` simulados por defecto (~100 sincronizaciones por segundo de simulación).

Para acelerar, en este orden:

1. `--sumo-updates=0.1` en el comando del EVA (mayor impacto: ~10× menos sincronizaciones TraCI; posiciones cada 0.1 s sim, de sobra para el visor).
2. Subir el slider a 20–30 fps.
3. `APP_STEP_LENGTH` mayor (2–5) en el compose de integración: más rápido, pero el movimiento se ve a saltos.
4. Quitar carga a ns-3 si no se necesita en esa corrida: `--met-sup=false`, sin `--csv-log`.

Nota: fluidez (25-ago)

El frontend **interpola el movimiento entre frames** (posición y rumbo, con detección de teletransportes) y el default pasó a `APP_STEP_LENGTH=0.5`. La interpolación refresca **solo la capa de vehículos** y adapta su ritmo a la flota (60 fps hasta 150 vehículos → ~11 fps hasta 1500; por encima se desactiva).

Advertencia

Con flotas grandes hay dos límites distintos. (1) *Render (frontend)*: resuelto con lo anterior. (2) *ns-3 (VaN3Twin)*: el coste por segundo simulado crece con el número de nodos (CAM/CPM broadcast $\approx O(N^2)$ recepciones) y **ns-3 es mono-hilo** — más CPUs de Docker no lo aceleran. Si ns-3 no da abasto, los frames del visor llegan tarde; la interpolación mide el intervalo real y estira el movimiento (fluido, pero más lento que tiempo real). Palancas: `--sumo-updates=0.1`, `--penetrationRate=0.5` (menos nodos radio sin quitar vehículos).

de SUMO), `--met-sup=false`, sin `--csv-log`. Guía de `APP_STEP_LENGTH`: flotas pequeñas (<50) $\rightarrow 0.2-0.5$; flotas grandes $\rightarrow 0.5-1.0$ (menos frames y menos payload por segundo simulado; la interpolación aporta la suavidad).

8 Qué ofrece el visor durante la corrida

Vehículos 3D procedurales a su tamaño y rumbo reales; calles coloreadas por nivel de servicio (LOS A–F) según la densidad en vivo; semáforos sincronizados con las fases de SUMO; **clic derecho** sobre un vehículo, calle o semáforo abre el inspector (velocidad, CO₂, combustible, ruido, tiempo perdido, carril, progreso de ruta...); panel de *Históricos* con series en vivo (vehículos por tipo, CO₂ de flota, tiempo de viaje, esperas en semáforo); capa de calles concurridas; presets de iluminación (amanecer/día/atardecer/noche) y cámara 2D/3D con auto-órbita.

Nota

El visor muestra la *verdad terreno* de SUMO (todos los vehículos). La *percepción por radio* (CAMs recibidos por cada nodo) se analiza con los `.pcap/CSV` de ns-3 (Wireshark: filtros `gnw`, `btpb`, `its`) o con el GEO SUMO WEB de la práctica (:8090). Son vistas complementarias.

9 Editar código y recompilar

El árbol vive en el volumen `van3twin_ns3-workspace` y se **edita directamente ahí**, dentro del contenedor (`docker compose exec van3twin bash + nano/vim`, o VS Code con la extensión *Dev Containers* adjuntada al contenedor `van3twin`; ruta `/home/vanet/VaN3Twin/ns-3-dev`).

Después de editar, según el tipo de fichero:

Qué editaste	Qué hacer después
C++ (<code>src/**/*.*cc</code> , <code>*.h</code>)	<code>./ns3 build</code> (incremental) $\rightarrow$ relanzar la simulación (Pasos A–B–C)
CMakeLists.txt o ejemplo nuevo	<code>./ns3 build</code> (re-ejecuta cmake solo) $\rightarrow$ relanzar
Escenario SUMO (<code>.rou.xml</code> , <code>.sumocfg</code> , <code>.net.xml</code>)	Nada que compilar: relanzar la simulación. Si cambió el net , también <code>up -d</code> del stack SUMO-GEO (el backend carga la geometría al arrancar)
Backend SUMO-GEO (Mac, <code>backend/</code>)	<code>docker compose --profile visor build backend + up -d</code> (desde <code>~/van3twin-docker</code>)
Frontend SUMO-GEO (Mac, <code>frontend/</code>)	Nada: recargar el navegador (nginx lo sirve montado). Si el cambio no aparece: recarga dura Cmd+Shift+R (descarta el <code>app.js</code> cacheado)

Advertencia

Copia de seguridad: las ediciones viven *solo* en el volumen — `docker compose down -v` las borra. Para versionar, el árbol tiene git dentro (`cd ~/VaN3Twin/ns-3-dev && git add -A src/automotive src/traci && git commit`); o exporta ficheros puntuales a `~/results` (visible en el Mac). **Nunca** montar el árbol como bind mount del Mac ni compilar desde una carpeta APFS: los pares de ficheros que solo difieren en mayúsculas (`ActionID/ActionId`, `BOOLEAN/boolean...`) colisionan y rompen la compilación.

10 Replay offline: mensajes V2X desde los .pcap

Reproduce una corrida **grabada** en el visor 3D con el intercambio de mensajes real: pulso radial en cada transmisión, arco TX→RX por cada recepción (color por tipo: CAM azul, CPM verde, DENM rojo) y, con **clic izquierdo sobre un pulso**, el **contenido ASN.1 completo** del mensaje más la lista de receptores. No necesita SUMO ni TraCI: la movilidad se reconstruye de los encabezados GeoNetworking y los mensajes se decodifican con los **.asn** oficiales ETSI del árbol de VaN3Twin.

❖ DENTRO DEL CONTENEDOR **van3twin** — tras la corrida

```
cp ~/VaN3Twin/ns-3-dev/v2v-EVA-*.pcap ~/results/
```

■ EN EL MAC (PC local)

```
# nada mas que hacer: el backend detecta los ficheros nuevos de ~/results
# automaticamente (al pulsar Replay o al consultar la API) y recarga el indice
curl localhost:8000/api/replay/info # totales por tipo y PDR por par
```

✓ EN EL NAVEGADOR

Los controles viven en el panel **Replay V2X** (columna izquierda, debajo del panel SUMO · GEO): botón *Replay pcap*, línea de tiempo con *seek*, botón play/pausa, y botones ◀/▶ (paso atrás/adelante: salta un frame y pausa, con los mensajes congelados en pantalla; clic sobre cualquier **pulso o arco** abre la **disección por capas** estilo Wireshark — 802.11 → GeoNetworking → BTP-B → ITS/ASN.1, secciones plegables con scroll), selector de vehículo emisor (All/vehX; al elegir uno, el vehículo se resalta con un anillo ámbar en el mapa) y filtros CAM/CPM/DENM. Debajo, el panel **PHY 802.11p** con las métricas de capa física de la corrida: banda/canal/BW/modulación y potencia TX (config. del EVA), latencia TX→RX medida (media/p95/máx), rango de cobertura observado, PER global (RX logradas vs. esperadas según coexistencia), PDR mejor/peor par, tasas por tipo y utilización de canal estimada por airtime. Los **enlaces llevan etiqueta con la distancia en metros** (visible en paso/pausa o con pocos arcos) y el popup lista cada receptor con su distancia. La potencia RX no viene en los pcap (sin radiotap), pero un **signal-rx.csv** en **~/results** (callbacks Signal-Info del EVA; formato **rx,tx,t_ms,rsi[,snr]**; código en **Practica_Replay_V2X.pdf**) se ingiere solo y añade el RSSI a etiquetas, popup y panel PHY. Los paneles se auto-ocultan a su barra de título; el pin del título los fija.

Nota

El uso didáctico completo — incluido el análisis de los mismos pcaps con Wireshark y el contraste entre ambas vistas — está en la práctica aparte: **Practica_Replay_V2X.pdf**.

Nota: notas técnicas

Movilidad desde la posición del emisor en cada paquete (~10 Hz; vehículos que nunca transmiten no aparecen). El pool de nodos de ns-3 se reutiliza entre vehículos: el replay lo resuelve por segmentos temporales decodificando los CAM. Arcos RX muestreados a ≤ 400 por frame. GN Beacons excluidos. Decodificación validada (SUMO 1.12 + VaN3Twin ago-2026): CAM (EN 302 637-2), CPM v2 (CPM-a11.asn — no usar la variante TR 103 562+ISO 19091: tarda minutos en compilar), DENM.

11 Solución de problemas

Síntoma	Causa	Solución
Visor en negro, sin mapa	Backend caído o sin responder	<code>docker compose logs backend</code> (desde <code>~/van3twin-docker</code>); <code>curl localhost:8000/api/health</code>
<code>waiting for 2 clients</code> eterno	El visor no conectó	Recargar <code>localhost:8081</code> ; ver logs del backend
Solo 1 client connected y visor en negro	Backend conectando a un puerto sin SUMO	Verificar <code>APP_SUMO_PORT_SCAN=10</code> ; ¿corrida vieja ocupando puertos? → Paso A
<code>Invalid command-line arguments</code>	Flag inexistente o binario sin recompilar	<code>./ns3 build</code> tras editar; revisar el nombre del flag
La simulación arranca sin esperar al visor	Faltó <code>--num-traci-clients=2</code>	Relanzar con el flag
Mapa base de Cuenca en vez de Turín	Backend viejo sin el fix de <code>geo.py</code>	<code>--profile visor build backend + up -d</code>
Vehículos congelados	El visor se cerró/recargó a mitad	Paso A → B → C
Error de versión TraCI en logs del backend	Pines ≠ SUMO del contenedor	<code>requirements-van3twin.txt</code> debe pinar la versión de <code>sumo --version</code> (hoy 1.12.0)
<code>g++ ... Killed</code> al compilar	RAM de la VM Docker insuficiente	Docker Desktop → Resources: RAM ≥ 1.5×CPUs (GB)
Error con <code>MakeTupleChecker</code> al compilar	Se compiló desde una carpeta APFS del Mac	Compilar solo en el volumen (§9)
El visor se comporta “como antes” tras un cambio del frontend	<code>app.js</code> cacheado por el navegador	Recarga dura: Cmd+Shift+R. El <code>nginx</code> ya envía <code>no-store</code> : tras una recarga dura no vuelve a pasar

11.1 Comandos de diagnóstico, en detalle

Todos se ejecutan desde `~/van3twin-docker` salvo que se indique lo contrario.

Estado de los contenedores.

```
docker compose --profile visor ps -a
```

Lista los tres servicios (`van3twin`, `backend`, `frontend`) con su STATUS. `Up` = corriendo; `Exited (N)` = terminó o crasheó → mirar sus logs. Sin `--profile visor`, los servicios del visor no aparecen aunque existan.

Contenedores huérfanos ocupando puertos.

```
docker ps -a | grep -iE 'backend|frontend'
```

Si aparecen `sumo_geo-backend-1/sumo_geo-frontend-1` (del `compose` separado antiguo) retienen los puertos 8000/8081 y el stack unificado no puede arrancar: `docker rm -f sumo_geo-backend-1 sumo_geo-frontend-1`.

Logs del backend (la herramienta principal).

```
docker compose logs backend | tail -20
```

Qué buscar: `[sumo_bridge] SUMO respondió en 34XX` = conexión TraCI correcta aunque el puerto haya derivado; `Connection refused` = no hay SUMO esperando (falta el Paso B o el flag `--num-traci-clients=`; `traceback` de Python al arrancar = fallo cargando la geometría (ruta `APP_NET_FILE`, volumen); error de versión TraCI = pines de `requirements-van3twin.txt` distintos del SUMO del contenedor.

Salud y metadatos del backend.

```
curl http://localhost:8000/api/health # esperado: {"status":"ok","mode":"remote"}
curl http://localhost:8000/api/meta   # esperado: center ~ [7.66, 45.06] (Turin)
```

Sin respuesta = backend caído (ver logs). Si `center` sale $\approx [-79, -2.9]$ (Cuenca), el backend corre una imagen vieja sin el fix de `geo.py` → reconstruir con `--build`.

Procesos SUMO/ns-3 colgados (dentro del contenedor).

```
pgrep -a sumo # hay SUMO vivos de corridas anteriores?
pkill -f ns3-dev-v2v; pkill sumo # limpieza (Paso A)
```

Un SUMO huérfano retiene su puerto: la siguiente corrida deriva a 3401, 3402... (el backend escanea hasta 3410, pero conviene limpiar).

Verificar los parches en el volumen (tras un `down -v` o ante dudas).

```
docker compose exec van3twin grep -c "num-clients" src/traci/model/traci-client.cc # >= 1
docker volume ls | grep ns3-workspace # existe
```

En el navegador. Consola JS (Opt+Cmd+J en Chrome) y pestaña *Network*. “esperando al backend... (intento N)” en pantalla o `/api/meta` eternamente *pending* = backend sin responder; el WebSocket `/ws/live` con mensajes `frame` fluyendo = todo sano.

11.2 Cuándo usar `--build` (y cuándo no)

`up -d` recrea contenedores (aplica cambios del `compose`: variables, puertos, montajes). `--build` además **reconstruye la imagen**: solo hace falta cuando cambian ficheros que se *copian o instalan dentro* de la imagen.

Cambio	¿ <code>--build</code> ?	Comando
backend/app/*.py, requirements-van3twin.txt, Dockerfile.van3twin	Sí	<code>docker compose --profile visor build backend + up -d</code>
Variables/puertos/montajes del <code>compose</code>	No	<code>docker compose --profile visor up -d</code>
Código ns-3 (vive en el volumen)	No	<code>./ns3 build</code> dentro del contenedor
frontend/ de SUMO_GEO (montado en <code>nginx</code>)	No	recargar el navegador
Escenarios SUMO (<code>.rou.xml</code> , <code>.sumocfg</code>)	No	relanzar la simulación
Dockerfile de van3twin (imagen ns-3)	Sí	<code>docker compose build</code> (1–3 h; el volumen sigue tapando el árbol nuevo)

Regla rápida: si el fichero aparece en un `COPY/RUN` de un `Dockerfile` → `--build`; si llega por volumen o variable de entorno → basta `up -d` (o nada).

Advertencia

Si un build tarda demasiado: el del backend toma ~2–5 min la primera vez (descarga de `python:3.11-slim` + wheels de pip) y segundos después (caché). Si en la salida aparecen `gRPC`, `apt` del `toolchain` o compilación de `ns-3`, se está reconstruyendo la imagen grande de `van3twin` por error: `Ctrl+C` (seguro — las capas cacheadas se conservan) y usar la forma en dos pasos: `build backend` primero, `up -d` después. Por eso el manual evita `up -d --build`, que según la versión de `compose` puede arrastrar dependencias al `build`. **Nunca ejecutar `docker compose build` sin argumento en este proyecto:** construye todos los servicios, `van3twin` incluido. Para máxima seguridad al levantar: `docker compose --profile visor up -d --no-build`. Cancelar una reconstrucción accidental es seguro: la imagen existente

solo se reemplaza cuando el build termina.

12 Cierre de sesión

■ EN EL MAC (PC local)

```
cd ~/van3twin-docker
docker compose --profile visor stop      # pausa todo (van3twin + visor)
# o: docker compose --profile visor down # elimina contenedores; el volumen persiste
```

Advertencia

`docker compose down -v` en `van3twin-docker` borra el volumen con el build de `ns-3`. No usarlo salvo que se quiera reconstruir todo desde la imagen.

13 Resumen de comandos

Primera ejecución (construye la imagen del backend; ~2–3 min)

■ EN EL MAC (PC local)

```
# si quedaron contenedores del antiguo compose separado, retirarlos una unica vez:
docker rm -f sumo_geo-backend-1 sumo_geo-frontend-1 2>/dev/null

cd ~/van3twin-docker
docker compose --profile visor build backend  # SOLO la imagen del backend (~2-5 min)
docker compose --profile visor up -d         # van3twin + visor
docker compose exec van3twin bash
```

⚙ DENTRO DEL CONTENEDOR `van3twin`

```
pkill -f ns3-dev-v2v; pkill sumo; sleep 1
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --met-sup=true --sumo-updates
=0.1 --num-traci-clients=2"
# -> "waiting for 2 clients... client connected" y se queda esperando
```

🔗 EN EL NAVEGADOR

Abrir <http://localhost:8081> → arranca el lockstep.

Resto de ejecuciones

■ EN EL MAC (PC local)

```
cd ~/van3twin-docker
docker compose --profile visor up -d      # solo si no esta ya arriba
docker compose exec van3twin bash
```

⚡ DENTRO DEL CONTENEDOR van3twin

```
pkill -f ns3-dev-v2v; pkill sumo; sleep 1
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --met-sup=true --sumo-updates
=0.1 --num-traci-clients=2"
```

✓ EN EL NAVEGADOR

Recargar <http://localhost:8081>.

Nota

Nueva corrida en la misma sesión: Ctrl+C → pkill → relanzar → recargar la página. Al terminar la jornada: `docker compose --profile visor stop`. Tras editar código: tabla del §9 (C++ → `./ns3 build`; backend → `--profile visor build backend + up -d`; escenario SUMO → solo relanzar).

Documentos relacionados: `GUIA_INTEGRACION_SUMO_GEO.md` (diseño original de la integración); `CONTEXTO_VAN3TWIN.md` y `CONTEXTO_SUMO_GEO.md` (estado técnico de cada proyecto y correcciones aplicadas sobre la guía); `MANUAL_SIMULACION_VAN3` (versión Markdown de este manual).